

Schema validation and dead-letter assertions

```
1 def test_wrong_schema_version_goes_to_dlq(mock_kafka_client):
2     event = {
3         "event_id": "123",
4         "event_type": "customer.created",
5         "event_source": "test_agent",
6         "event_time": datetime.now(timezone.utc).replace(tzinfo=None).isoformat(),
7         "entity_id": "cust_1",
8         "schema_version": "1.0", # Invalid
9         "payload": {}
10    }
11
12    mock_kafka_client.publish("acis.customers", event)
13    args, kwargs = mock_kafka_client._producer.send.call_args
14    assert args[0] == "acis.dlq"
15    assert b"publish_validation_failed" in kwargs["value"]
```

Invoice lifecycle trace and stage hand-off

```
1 # Instantiate agents sharing the same kafka mock
2 csa = CustomerStateAgent(kafka_client=kafka)
3 ppa = PaymentPredictionAgent(kafka_client=kafka)
4 rsa = RiskScoringAgent(kafka_client=kafka)
5 col = CollectionsAgent(kafka_client=kafka)
6
7 # -- T0: Inject invoice.created --
8 inv_event = Event(
9     event_id=f"evt_inv_{seq:06d}",
10    event_type="invoice.created",
11    event_source="ScenarioGeneratorAgent",
12    event_time=datetime.now(timezone.utc).replace(tzinfo=None),
13    entity_id=customer_id,
14    correlation_id=corr_id,
15    schema_version="1.1",
16    payload={
17        "customer_id": customer_id,
18        "invoice_id": invoice_id,
19        "amount": 50_000.0 + seq,
20        "total_amount": 50_000.0 + seq,
21        "due_date": (datetime.now(timezone.utc).replace(tzinfo=None) + timedelta(days=30)).isoformat(),
22        "issued_date": datetime.now(timezone.utc).replace(tzinfo=None).isoformat(),
23        "status": "pending",
24    },
25    metadata={},
26 )
27 t0 = time.monotonic()
28 csa.process_event(inv_event)
```

```
1 # -- T2: Feed to PaymentPredictionAgent --
2 metrics_event = Event.model_validate(metrics_entry["raw"])
3 metrics_event = metrics_event.model_copy(update={"correlation_id": corr_id})
4 ppa_offset = len(kafka.published)
5 ppa.handle_event(metrics_event)
6
7 # -- T3: Find payment.risk.predicted --
8 pred_entry = None
9 for e in kafka.published[ppa_offset:]:
10     if e["event_type"] == "payment.risk.predicted":
11         pred_entry = e
12         break
13 if not pred_entry:
14     return None
15 t3 = pred_entry["ts"]
16
17 # -- T4: Feed to RiskScoringAgent --
18 pred_event = Event.model_validate(pred_entry["raw"])
19 pred_event = pred_event.model_copy(update={"correlation_id": corr_id})
20 rsa_offset = len(kafka.published)
21 rsa.handle_event(pred_event)
22
23 # -- T5: Find risk.scored --
24 risk_entry = None
25 for e in kafka.published[rsa_offset:]:
26     if e["event_type"] == "risk.scored":
27         risk_entry = e
28         break
29 if not risk_entry:
30     return None
31 t5 = risk_entry["ts"]
32
33 # -- T6: Feed to CollectionsAgent --
34 risk_event = Event.model_validate(risk_entry["raw"])
35 risk_event = risk_event.model_copy(update={"correlation_id": corr_id})
36 col_offset = len(kafka.published)
37 col.process_event(risk_event)
```

P95 pipeline latency measurement

```
1         t_start = time.monotonic()
2         injection_timestamps[corr_id] = t_start
3         pub_offset = len(kafka.published)
4
5         csa.process_event(invoice_event)
6
7         for e in kafka.published[pub_offset:]:
8             if (
9                 e["event"].get("event_type") == "customer.metrics.updated"
10                and e["event"].get("correlation_id") == corr_id
11            ):
12                ppa.handle_event(Event.model_validate(e["event"]))
13                break
14
15         for e in kafka.published[pub_offset:]:
16             if (
17                 e["event"].get("event_type") == "payment.risk.predicted"
18                and e["event"].get("correlation_id") == corr_id
19            ):
20                rsa.handle_event(Event.model_validate(e["event"]))
21
22         # Compute latencies
23         risk_scored_by_corr: Dict[str, float] = {}
24         for entry in kafka.published:
25             evt = entry["event"]
26             if evt.get("event_type") == "risk.scored":
27                 corr = evt.get("correlation_id")
28                 ts = entry["timestamp"]
29                 if corr and (corr not in risk_scored_by_corr or ts < risk_scored_by_corr[corr]):
30                     risk_scored_by_corr[corr] = ts
31
32         latencies_ms = [
33             (risk_scored_by_corr[corr] - t_inject) * 1000.0
34             for corr, t_inject in injection_timestamps.items()
35             if corr in risk_scored_by_corr
36         ]
37
38         completion_rate = len(latencies_ms) / self.NUM_EVENTS
39         assert completion_rate >= 0.50, (
40             f"Only {len(latencies_ms)}/{self.NUM_EVENTS} ({completion_rate:.1%}) "
```

```
1     )
2
3     sorted_lat = sorted(latencies_ms)
```

Hard-failure recovery assertion

```
1  def test_hard_failure_triggers_restart(self):
2      kafka = _make_kafka()
3      sha = _make_sha(kafka)
4
5      # Pre-register the agent
6      sha._states[self.AGENT_ID] = AgentRecoveryState(
7          agent_id=self.AGENT_ID,
8          agent_name=self.AGENT_NAME,
9          status=AgentStatus.HEALTHY.value,
10     )
11
12     # Inject TIMEOUT fault and measure recovery latency
13     event = _make_timeout_event(self.AGENT_NAME, self.AGENT_ID)
14     t_fault = time.perf_counter()
15     sha.process_event(event)
16     t_recovery = time.perf_counter()
17
18     recovery_ms = (t_recovery - t_fault) * 1000.0
19
20     # Assertions
21     restart_count = _count_published(kafka, SystemEventType.AGENT_RESTART_REQUESTED.value)
22     recovery_count = _count_published(kafka, SystemEventType.RECOVERY_TRIGGERED.value)
23
24     logger.info(
25         f"\n Hard Failure Test:\n"
26         f"    recovery.triggered events : {recovery_count}\n"
27         f"    restart.requested events  : {restart_count}\n"
28         f"    recovery latency          : {recovery_ms:.3f} ms"
29     )
30
31     assert recovery_count >= 1, (
```